Klaus x DevOps

Anders Mouanga `mouanga@kth.se`
Erik Söderlund `eriso@kth.se`

October 27, 2026

1 Introduction

This project adds several DevOps features to a fork of Jonas Haag's `klaus` project (https://github.com/jonashaag/klaus). `klaus` is a web-based Git viewer that is designed to be easy to set up and use. It can be run using Docker, or pip, among others. To narrow down the scope of this project, we have decided to limit our version of `klaus`,appropriately called `klaus-clone`, and hosted on GitHub (https://github.com/superderker/klaus-clone), to run on Docker only.

2 Features

2.1 Continuous integration

The main branch is protected from direct pushes, and must be merged to using pull requests. On pull requests and merges to main, the following CI pipeline is run. Note that CI passing is mandatory for a branch to even push to main.

- Ruff: a linter that verifies that the proposed code maintains the proper style.
- mypy: a tool that looks at the Python code's type annotations and makes sure it all makes sense.
- Bandit: another linter, scans the code for security vulnerabilities.
- Gitleaks: scans pull requests to see if any secrets (like API tokens) have been leaked.

These tools are used as they are fast (especially Ruff), industry-standard DevOps software that are suitable for Python development. Together, they cover code style, correctness and safety. Furthermore, we use a subset of the tests from the original `klaus`.

2.2 Terraform and Continuous Deployment

On every push to main, the workflow builds and scans the Docker image, then a terraform job authenticates to AWS through an OIDC role, loads the shared state from an S3 bucket, and runs plan and apply so the real infrastructure matches the .tf files. The state in S3 is what lets each fresh GitHub runner know what already exists, so Terraform only changes what differs instead of recreating everything. Finally, the deploy job takes the server's IP from terraform output, SSHes in, and runs git pull and docker compose up -d to roll out the new container. Old containers are cleared to free up disk space. Note that CD is triggered by CI, and it builds whichever code passed the triggering CI run (instead of the latest code in the main branch). This is to prevent the race condition where broken code is pushed to main between CI's completion and CD's build and deployment.

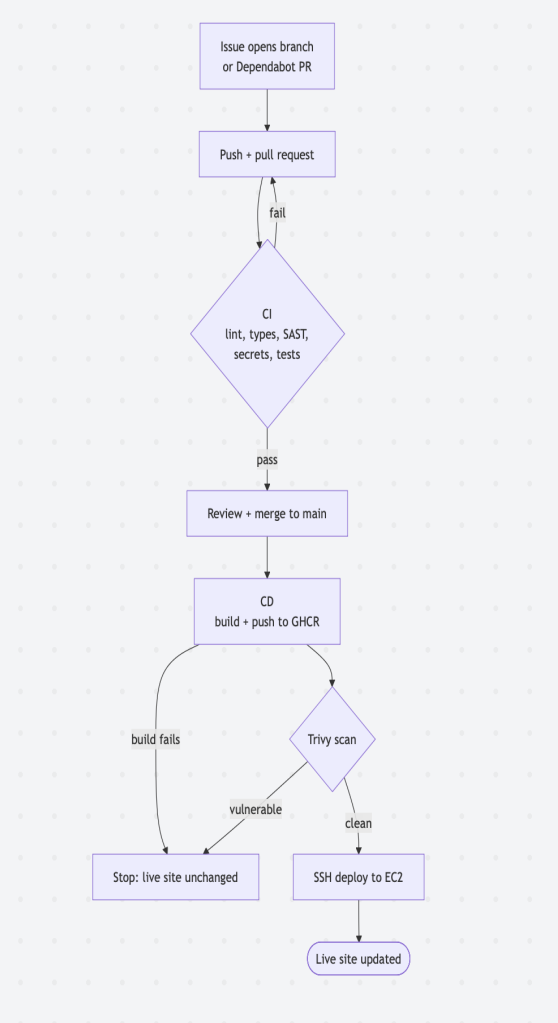


Figure 1: The pipeline from pull request to live deployment.

2.3 Runtime configuration

Docker Compose defines two containers on a private network: Klaus and an Nginx reverse proxy that is the only service exposed to the internet. The credentials file is provided as a Docker secret, and the repositories are mounted read-only.

2.4 Secrets

No secrets are stored in the repository or in the Docker image. The authentication file for Klaus (`htdigest.txt`) is created by hand on the server (aws hosted version) and mounted into the container as a Docker secret, so it is only visible to the application at runtime. It is excluded from Git and from the Docker build context. The deployment credentials, the server address and the SSH private key, are stored as encrypted GitHub Actions secrets and are only available to the CD workflow, while

publishing to the container registry uses the temporary token that GitHub generates for each run.

2.5 Dependabot

In addition, Dependabot is used to monitor software packages for security vulnerabilities and outdated versions. If Dependabot finds an outdated or insecure package, it automatically opens a pull request and tries to fix the issue without breaking existing code. However, manual intervention is still required to trigger CD.

2.6 CodeRabbit

CodeRabbit runs next to the CI pipeline as an additional verification step to ensure code quality. It scans pull requests and generates comments about the code.

3 Critical evaluation

A limitation of our DevOps pipeline is the tooling coverage; it does not catch all bugs or issues (and by Rice's theorem, doing so is even impossible!). This could have been mitigated by adding even more tools, but this could lead to notification blindness, longer development time, and a project with a larger scope than intended. All updates cause minimal downtime because only one server is in use. Using e.g. blue-green deployment would have introduced unnecessary complexity (and cost) to a project of this size. Furthermore, having only one server introduces a single point of failure and so the redundancy is very low. The decision to do so was made based on its lower complexity and cost.

4 Use of AI tools

Artificial intelligence was used as part of the CI pipeline through the use of CodeRabbit. Furthermore, it was also consulted for help with ideas for the code and syntax.